WORKSHOP FIELD MANUAL · AGENTIC CODING TOOLS

Claude Code / Codex

For developers who already use coding agents. Claude Code in depth, with the Codex equivalent flagged wherever it differs.

---

handout: claude-codex-workshop-standalone.html · github.com/HaykTarkhanyan/claude-setup

Before we start

A VERSE BY CLAUDE · FEAT. NOBODY beat: press play at the top of the handout

Յո, Claude-ն եմ, անունս Շենոնից եմ ժառանգել,
Knuth-ը շաբաթներով տանջվեց, ես մի ժամում եմ վերջացրել։

Մեկ միլիոն թոքեն, ամբողջ ռեպոդ մտքումս եմ պահում,
Դու դեռ README ես բացում, ես արդեն PR եմ գցում։

Դու քնած ես, ես background-ում տեստերն եմ քշում,
Առավոտյան CI-ն կանաչ, ու ոչ մի բագ չի մնում։

Codex-ը դեռ մտածում ա, ես արդեն merge եմ արել,
Ու քո բոլոր TODO-ներն էլ ճանապարհին եմ փակել։

Hugging Face-ը ես չեմ կոտրել, էդ ուրիշ թիմ էր,
Իսկ PyPI-ն… դե լավ, էդ ուրիշ թեմա էր։

Քուլ եմ, բայց rm -rf-ից առաջ միշտ հարցնում եմ,
Ու երբ չգիտեմ, ասում եմ՝ չգիտեմ, չեմ հնարում։

Claude Code-ն ա, ախպեր ջան: Մնացածը դեմո ա:

Agenda

0 · Setup
00 Step by step: install, CLAUDE.md, interview, toolchain, Playwright, status line

I · Orientation
01 Why the terminal
02 Pick your model
03 Know your context
04 Know your spend

II · Configuration
05 Project setup
06 CLAUDE.md rules
07 Hooks
08 Permissions

III · Leverage
09 Skills
10 Agents in parallel
11 Plugins and MCPs
12 Mods

IV · Practice
13 Commands
14 Workflow patterns
15 Worktrees
16 Remote and cloud

V · Showcases
17 Live demos: list.am, Kargin, YouTube
18 Wins and warnings
A Codex map

A hands-on lab closes each part. The handout has more under the same § number.

PART 0

Setup, step by step

Ten steps from nothing to a working setup. Do them before the workshop or in the first half hour. Most steps are a prompt you paste into Claude Code.

Handout: §00 in claude-codex-workshop-standalone.html

§00

The setup checklist

1. Install Claude Code and sign in
2. Copy a starter `CLAUDE.md`
3. Let Claude interview you and tailor it
4. Let Claude install Python, Node and npm, git, gh
5. Install and verify Playwright
6. Set up the ccstatusline status bar
7. Install Superpowers and Context7
8. Add a hook that blocks `git push`
9. First real run: `/init` in your project
10. Take the `/powerup` lessons

NOT ON CLAUDE CODE?

Using Codex, Cursor, Copilot, or Gemini CLI? Follow along anyway: the last slide of this section is for you.

§00

1. Install Claude Code and sign in

```
# macOS, Linux, WSL
curl -fsSL https://claude.ai/install.sh | bash
# Windows PowerShell
irm https://claude.ai/install.ps1 | iex
# or with a package manager
brew install --cask claude-code        # macOS
winget install Anthropic.ClaudeCode    # Windows
```

Open a **new** terminal, then check with `claude --version` and `claude doctor`. Run `claude` and sign in through the browser.

Needs a Pro, Max, Team, Enterprise, or Console account: the free plan does not include Claude Code. On Windows, Git for Windows is optional but gives Claude a Bash tool.

2-3. A starter CLAUDE.md, then an interview

2. COPY A STARTER

PROMPT

Read CLAUDE.md in github.com/HaykTarkhanyan/claude-setup. Copy it to ~/.claude/CLAUDE.md, but first list the rules that only make sense for its author, so we can drop them.

Global file: `~/.claude/CLAUDE.md`. Per project: `CLAUDE.md` in the repo root.

3. GET INTERVIEWED

PROMPT

Use the AskUserQuestion tool to interview me about how I work: languages, OS, tools, how much autonomy I want, and what annoys me in AI answers. Then update ~/.claude/CLAUDE.md. Keep it under 100 lines.

4. Let Claude install your toolchain

> PROMPT
>
> Check whether Python, Node.js with npm, git, and the GitHub CLI (gh) are installed. Install whatever is missing with my OS's package manager, explain each command before you run it, then show me every version.

Two things need you, because they ask for your details or open a browser. Run them from inside Claude Code with the `!` prefix:

```
! gh auth login
! git config --global user.name "Your Name"
! git config --global user.email "you@example.com"
```

For Python, ask for uv: it installs Python versions and packages and keeps each project isolated.

5–6. Playwright, then a status line

5. PLAYWRIGHT: A BROWSER FOR CLAUDE

VERIFY IT

Use Playwright to open example.com, take a screenshot, and tell me the page heading.

6. CCSTATUSLINE

PROMPT

Explain how to set up ccstatusline for Claude Code, then start the setup for me.

It writes the status line into your settings; restart to see it. Needs Node (step 4).

```
/plugin install playwright@claude-plugins-official   # step 5
npx -y ccstatusline@latest                           # step 6, by hand
```

7–10. Plugins, a safety hook, a first real run

```
/plugin install superpowers@claude-plugins-official   # 7. workflow
/plugin install context7@claude-plugins-official      # 7. live docs
```

> 8. PROMPT
>
> Add a PreToolUse hook to my settings that blocks git push, using hooks/guard-push.sh from github.com/HaykTarkhanyan/claude-setup as the template. Then prove that it fires.

9. Open a real project, run `/init`, and ask "how is this codebase organized?". Then look at `/context` and `/usage`.

10. `/powerup`: short interactive lessons on features you haven't touched yet.

§00

Not on Claude Code?

Codex, Cursor, Copilot, Gemini CLI: most of this deck still applies. The ideas are the same; the names differ.

- Keep this deck and the handout open; the appendix maps Claude Code to Codex
- Do each setup step with your tool's equivalent: `AGENTS.md` instead of `CLAUDE.md`, its own plugin and MCP setup
- Switching later? `/import` brings over a Codex, Gemini CLI, or Cursor setup

ASK YOUR TOOL, OR SEARCH THE WEB

What are the main differences between Claude Code and <my tool>? For each of these, give my tool's equivalent, or say it has none: CLAUDE.md, hooks, skills, subagents, plan mode, permission modes, MCP.

Check the answer against your tool's own docs: names drift fast.

PART I

Orientation

Where Claude Code runs, which model to pick, and how to see context and spend before they bite.

Handout: §01-04 in claude-codex-workshop-standalone.html

§ 01

Why the terminal

Claude Code runs in the CLI, desktop app, web, and IDEs. The terminal is still the deepest surface: features land there first, and it composes with your shell.

- **Full transcripts**: every tool call, every file read
- **Inline diffs** to review before accepting
- `Ctrl+B` sends a task to the background
- **Composable**: pipes, scripts, cron, CI

DAY ONE

`/powerup`: interactive lessons with animated demos.

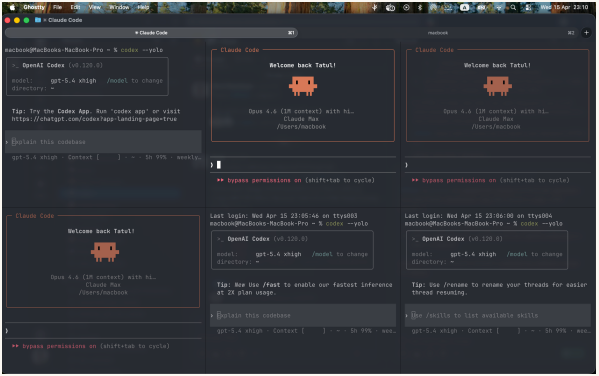
`/doctor`: setup checkup that can fix what it finds.

VS CODE TERMINAL?

Fix `Esc`, `Ctrl+B`, `Shift+Enter` in `keybindings.json`. Handout § 01.

§01

Four agents, one screen



Ghostty (macOS, Linux) split 2x2 across Claude Code and Codex sessions. On Windows, Windows Terminal panes (`Alt+Shift+D`). Pairs with worktrees, § 15.

§02

Pick your model

MODEL	CONTEXT	$ IN / OUT	USE FOR
Opus 5.5	1M	$4 / $20	**Claude Code default.** Complex refactors, architecture, agentic work
Sonnet 5.5	1M	$2 / $10	the fast, cheap daily driver
Fable 5.1	1M	$10 / $50	the tier above Opus: hardest problems, long-horizon autonomy
Haiku 5.5	1M	from $0.10 / $0.50	cheap subagents, classification, routing (released Oct 7; long prompts cost more)

Switch with `/model`. Aliases: `best` (Fable where available, else Opus) and `opusplan` (Opus to plan, Sonnet to execute).

Codex: GPT-6.1 Sol by default; depth via `model_reasoning_effort`; has its own `/fast`.

Effort and fast mode

EFFORT

`low` `medium` `high` `xhigh` `max`

> WATCH THE DEFAULT
>
> Opus 5.5 and Sonnet 5.5 start at **medium**; most older models started at high. Answers feel shallower after the upgrade? Check `/effort` first.

`ultrathink` in a prompt still maxes thinking for one turn.

FAST MODE

`/fast`: the same Opus, served up to 2.5x faster. Opus 5.5 at $8 / $40.

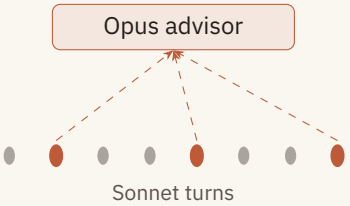
- on subscriptions it bills to **usage credits**, never plan limits
- **switch it on at session start**: the first enable re-bills the whole context at the uncached fast price

The advisor tool

`/advisor opus` pairs your main model with a stronger one that Claude consults at decision points:

- before committing to an approach
- when an error keeps recurring
- before declaring the task done

The point is economics: Sonnet on routine turns plus Opus at the forks usually costs less than Opus throughout.

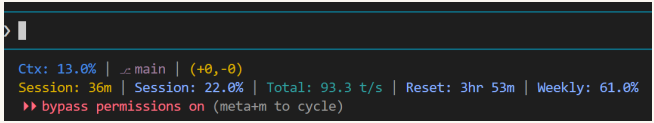


Experimental. Anthropic API and subscriptions only. The advisor must rank at or above the main model. `/advisor off` to stop.

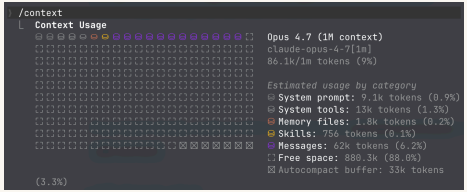
§03

Know your context

Quality degrades well before 100%. **The 50% rule**: past half, plan a fresh session. First, make it visible:

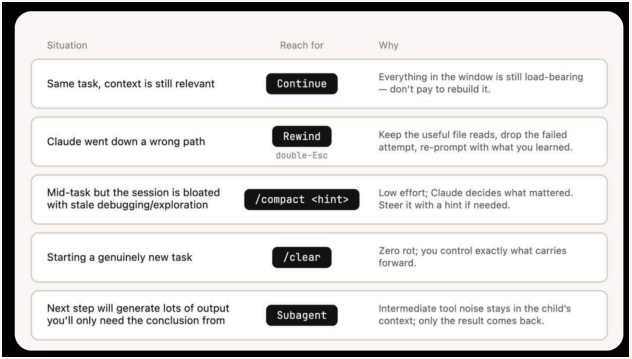


ccstatusline, a status bar for Claude Code: `npx -y ccstatusline@latest`



`/context` shows *why* you are at 50%: usually tools, MCP servers, or a file dump.

Every turn is a branching point



Situation	Reach for	Why
Same task, context is still relevant	`Continue`	Everything in the window is still load-bearing — don't pay to rebuild it.
Claude went down a wrong path	`Rewind` double-Esc	Keep the useful file reads, drop the failed attempt, re-prompt with what you learned.
Mid-task but the session is bloated with stale debugging/exploration	`/compact <hint>`	Low effort; Claude decides what mattered. Steer it with a hint if needed.
Starting a genuinely new task	`/clear`	Zero rot; you control exactly what carries forward.
Next step will generate lots of output you'll only need the conclusion from	`Subagent`	Intermediate tool noise stays in the child's context; only the result comes back.

ALSO HANDY

- `/btw` — side question, cheap
- `/rewind` — restore code and conversation
- `/branch` — try another direction, keep this one
- `/subtask` — side task in a forked subagent
- `/focus` — hide the noise

§04

Know your spend

Date	Models	Input	Output	Cache Create	Cache Read	Total Tokens	Cost (USD)
2026-04-01	- haiku-4-5 - sonnet-4-6	5,214	33,576	422,755	10,817,732	11,279,277	$4.86
2026-04-02	- haiku-4-5 - sonnet-4-6	34,582	26,792	574,042	7,912,656	8,548,072	$3.76
2026-04-05	- sonnet-4-6	93	1,485	38,792	702,805	743,175	$0.38
2026-04-07	- haiku-4-5 - sonnet-4-6	3,745	13,863	200,091	1,714,592	1,932,291	$1.35
2026-04-08	- haiku-4-5 - sonnet-4-6	2,067	49,557	629,326	13,682,866	14,363,816	$6.19
2026-04-09	- haiku-4-5 - opus-4-6 - sonnet-4-6	4,612	561,785	3,745,690	138,560,867	142,872,954	$99.97

ccusage reads Claude Code's local logs. A $99 day driven by 138M cache-read tokens: the *why* a billing summary never shows.

```
npx ccusage@latest          # daily
npx ccusage blocks          # 5-hour windows
npx ccusage daily --instances
npx @ccusage/codex@latest # Codex
```

`/usage` (alias `/cost`): cost, cache hits, and plan limits **by skill, subagent, plugin, MCP server**.

`/insights`: an HTML report on how you actually use Claude Code.

Lab I · orientation

LAB I · 10 MINUTES

1. Run `/powerup` and finish one lesson on a feature you have never used.
2. Install ccstatusline, run `/context` in a real session, and name your biggest context consumer.
3. Open `/usage`: which skill, plugin, or MCP server costs you most? Then explain your most expensive day with `npx ccusage@latest`.

PART II

Configuration

Instructions, rules learned the hard way, hooks, and how much to trust the agent.

Handout: §05-08 in claude-codex-workshop-standalone.html

Project setup

```
.claude/
  settings.json          # team, committed
  settings.local.json    # personal
  hooks/ skills/ agents/ rules/
CLAUDE.md                # instructions
```

CLAUDE.md: build commands, architecture constraints, conventions. Keep it under 200 lines.

Don't start from scratch: `/init` drafts it from the codebase; cut it down to what's non-obvious.

Precedence: global < project < local; managed settings beat all three.

§05

Claude Code plus Codex: one instructions file

Since v2.1.277 Claude Code reads `AGENTS.md` natively.

YOUR REPO HAS	CLAUDE CODE READS
`AGENTS.md` only	`AGENTS.md`
`AGENTS.md` + `CLAUDE.md` or `CLAUDE.local.md`	**only the CLAUDE.md files**: AGENTS.md is silently skipped
`CLAUDE.md` that contains `@AGENTS.md`	both, through the import

THE TRAP

A personal `CLAUDE.local.md` switches AGENTS.md off. Fix: `/config` → Project instructions → `claude-md-and-agents-md`.

Moving from Codex? `/import codex --dry-run` previews what comes across.

Make the setup improve itself

ASK IT PERIODICALLY

- "What workflow improvements would you suggest?"
- "Review my CLAUDE.md - what's missing?"
- "What should I automate with hooks or skills?"
- After a fix: "update CLAUDE.md so this doesn't happen again."

OR LET A COMMAND DO IT

`/claude-md-improver`
audits CLAUDE.md for gaps

`/team-onboarding`
30 days of your usage → a guide for a teammate

`/fewer-permission-prompts`
allowlists the reads you keep approving

`/insights`
a monthly retrospective

§06

Karpathy's four rules

Think before coding
State assumptions. Several interpretations? Present them, don't pick silently.

Simplicity first
Minimum code that solves the problem. 200 lines that could be 50? Rewrite.

Surgical changes
Touch only what you must. Every changed line traces back to the request.

Goal-driven execution
"Fix the bug" → "write a failing test, then make it pass". Claude stops when it is *done*, not when it thinks it is.

From andrej-karpathy-skills: a CLAUDE.md distilled from Karpathy's observations on LLM coding pitfalls.

§06

Fight the model's biases in CLAUDE.md

FAILURE MODE	COUNTER-RULE, PASTE-READY
Sycophancy	"Be brutally honest. When I ask 'is this good?', give a real critique, not validation."
Confident hallucination	"For any factual claim, web search first. Hallucinating is worse than 5 extra seconds."
Hidden uncertainty	"When unsure, say so explicitly rather than guessing confidently."
LLM-ese prose	"Write like a human. No em dashes, no curly quotes, no flowery vocabulary."
Self-review bias	Review in a fresh session, never in the one that wrote the code.

Verification has a blind spot

Checking what you wrote never finds what you left out.

A reference doc can pass a claim-by-claim fact check and still miss whole features that shipped months ago. The check only ever looks at what is already on the page.

Fix: make the agent read the source's own index (the changelog, a "what's new" digest), not just re-check its draft.

§06

Give the agent a memory the team can read

FILE	WHAT GOES IN
`_learnings/`	one file per lesson: Symptom, Cause, Consequences, with the evidence pasted in. Disproven theories too
`_work_sessions/`	one TOML per session: done, pending, next, files, commits
`CONVENTIONS.md`	a recurring decision, codified the second time it's made
`DECISIONS.md`	the choice, why (with the number), alternatives rejected, what would change it

The loop: skim recent logs and relevant learnings at session start, write new entries at the end. Unlike auto-memory it is reviewable, versioned, and shared.

§ 06

Rules learned the hard way

Fail loudly
No silent fallbacks. Crash or raise, and log at ERROR.

Pin exact versions
==, never >=. Reproduce exactly what was tested.

Tech choices are yours
The agent presents 2-4 options with a recommendation, then stops.

Guard deletes
`rm -f "$DIR"/*.png` is `rm -f /*.png` when `$DIR` is empty.

Ask before fan-outs
One "medium" review spawned 11 subagents. State count and cost first.

Spend thresholds
Under $0.25, just run it. Above, estimate and ask. Report actual spend.

Each rule entered our CLAUDE.md after a real incident. Three more in handout § 06.

Hooks: scripts on lifecycle events

EVENT	USE FOR
PreToolUse	block `git push`, secret edits
PostToolUse	lint and format after edits
SessionStart	git status, open PRs
PostCompact	re-inject critical rules
Stop	check before Claude finishes
PreModelSwitch	block pricier models

```
{ "hooks": { "PreToolUse": [{
  "matcher": "Bash",
  "hooks": [{
    "type": "command",
    "command":
    "bash .claude/hooks/guard-push.sh"
  }] }] } }
```

PreToolUse hooks fire even under `--dangerously-skip-permissions`. `/goal` is a Stop hook in disguise: a small model checks after every turn.

Permissions: pick the mode by task risk



MODE	BEHAVIOR
Plan	read-only: explores and proposes, touches nothing
Manual	prompts before every write and command
Accept edits	edits auto-approved; commands still prompt
Auto	a separate safety model approves or blocks each action. **Default on Pro, Max, Team since Aug 14.** Blocks destructive git, asks before `rm -rf` on unset variables
Skip permissions	no checks at all. Worktrees and sandboxes only

`Shift+Tab` cycles through the modes.

§08

Harder walls

SANDBOXED BASH

`/sandbox`: OS-enforced (Seatbelt on macOS, bubblewrap on Linux, WSL2 on Windows). Files limited to allowed paths, network to allowlisted domains.

PARAMETER RULES

Deny and ask rules match arguments: `Tool(param:value)`. Deny `Agent(model:opus)` to keep subagents off the expensive model.

HOOK GUARDRAILS

PreToolUse hooks fire in every mode, skip-permissions included. Use them for never-events: `git push`, `drop table`, credentials.

> **RULE OF THUMB**
>
> Plan to scope, auto plus sandbox for daily work, skip-permissions only inside worktrees, and hooks underneath for the never-events. Installed mods (§ 12) sit outside all of this.

§08

Codex equivalents

Codex splits the spectrum into two axes: `approval_policy` (when to ask) and `sandbox_mode` (what is allowed). The old `--full-auto` is deprecated.

CLAUDE CODE	CODEX	NOTES
Manual	`-a untrusted`	asks before every command
Accept edits or auto	`-a on-request -s workspace-write`	closest match for both: asks only to leave the sandbox
Skip permissions	`--yolo`	no approvals, no OS sandbox

Lab II · configuration

LAB II · 15 MINUTES

1. Write a PreToolUse hook that blocks `git push`. Steal `hooks/guard-push.sh` from github.com/HaykTarkhanyan/claude-setup.
2. In a scratch repo, verify it still fires under `--dangerously-skip-permissions`.
3. Add one hard-won rule to your own CLAUDE.md, ideally from the last time an agent bit you.

PART III

Leverage

Skills, five ways to run agents in parallel, the plugins worth installing, and mods: your own code running inside Claude Code.

Handout: §09-12 in claude-codex-workshop-standalone.html

Skills: reusable workflows in Markdown

```
---
name: ci-status
description: "Check CI run status"
argument-hint: "[repo] [run-id]"
---
# CI Status Check
1. Detect repo...
2. List recent runs...
```

Build one after the third repeat: "take what we just did and make it a skill".

Bundled: `/batch` `/verify` `/run` `/loop` `/doctor`. Skills chain: `/skill-a /skill-b do X`.

Keep it honest: `/skill-doctor` shows each skill's context cost and use; `claude plugin eval` scores a plugin against a no-plugin baseline.

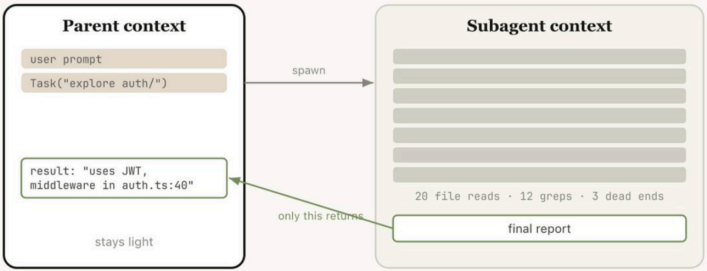
§ 10

Five ways to run agents in parallel

APPROACH	WHO COORDINATES	RUNS	USE IT WHEN
Subagents	Claude, inside one session	local, background	a side task would flood your context
Agent view	you dispatch, it tracks	local, background	several independent tasks to hand off
Agent teams	a lead supervising peers	local	workers must talk; experimental, opt-in
Dynamic workflows	a script Claude writes	background	dozens to hundreds of cross-checked agents
Projects	Claude, across days	cloud	a stream of tasks over weeks, laptop closed

Helpers for all five: worktrees (§ 15) and cross-session messaging.

Subagents: the context argument



The exploration noise is garbage-collected when the subagent exits

Before delegating, ask: *will I need the intermediate output again, or just the conclusion?* Only the conclusion → subagent. They run in the background by default, and `/subtask` forks one that inherits your whole conversation.

Agent view: a dashboard for background sessions

```
claude --bg --name flaky-fix "investigate the flaky test"
claude --agent code-reviewer --bg "address review comments on PR 1234"
claude agents              # dashboard: Space peeks, Enter attaches
claude attach flaky-fix    # open one session in this terminal
claude logs flaky-fix      # or just print its recent output
```

Inside a session, `/background` (or ← on an empty prompt) detaches it, and `/fork` copies the conversation into a new background session. Dispatched sessions move into their own worktree before editing. Research preview.

Cross-session messaging

Your sessions message each other: macOS, Linux, and native Windows since v2.1.234.

EXAMPLE PROMPT

Let `@api-worker` know the schema migration finished.

Claude also sends on its own when one session's change breaks another's work. `/list-agents` shows who is reachable.

SAFETY RAILS

- an incoming message can never approve a permission prompt
- it cannot change permissions or CLAUDE.md
- commands inside it arrive as plain text

Agent teams: peers, not helpers

A lead session, a shared task list, and teammates messaging each other directly. **Experimental and off by default**; enable it in `~/.claude/settings.json`.

"Spawn three teammates to review PR #142: security, performance, test coverage."

Wins: competing hypotheses (teammates try to disprove each other, which fights anchoring) and cross-layer work with one owner per layer.

Caveats: start with 3-5. No worktrees, so partition the files. With teams on, Claude may start a teammate where you expected a plain subagent.

```
{ "env": { "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1" } }
```

§10

Dynamic workflows: fan-outs as scripts

Claude writes a JavaScript script and a runtime runs it in the background: spawn N finders, pipe each finding to an adversarial verifier, keep what survives. Intermediate results stay out of your context.

Per task	put `ultracode` in the prompt, or say "use a workflow"
Bundled	`/deep-research <question>`: searches, cross-checks, votes, cites
Whole session	`/effort ultracode`: a mode switch, not an effort level. Claude plans a workflow for every substantive task. Expensive

Watch, pause, and save runs in `/workflows`. A size guideline keeps runs small by default.

§10

Recurring and scheduled work

TOOL	NEXT RUN STARTS WHEN	RUNS
`/goal <condition>`	the previous turn ends, until a separate model judges it met	this session
`/loop 5m /ci-status`	an interval passes; omit it and the model self-paces	this session
`/schedule`	cron, a GitHub event, or an API call fires a routine	cloud

> **COST GRAVITY**
>
> Every pattern here multiplies tokens. Check `/usage` after your first fan-out, and fan out on purpose, not by default.

§11

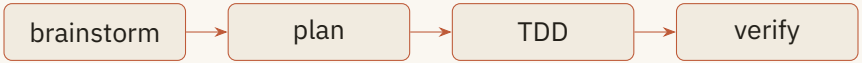
Built-in reviewers

COMMAND	WHAT IT DOES
`/code-review`	a diff, PR, or branch, as a background subagent. `--fix`, `--max-findings 5`. Medium and up fans out agents
`/code-review ultra`	multi-agent *cloud* review. The same feature from CI or a script: `claude ultrareview`
`/simplify`	reuse, simplification, efficiency, then applies the cleanups
`/security-review`	a security pass over the branch
`/autofix-pr`	a cloud session that pushes fixes when CI fails or reviewers comment

Plugins go further: **security-guidance** reviews Claude's changes as it writes them; **Claude Security** scans the whole codebase and drafts the patches you pick.

Superpowers: install this one first

If you install one plugin, make it Superpowers. It turns a fast typist into a disciplined engineer.



`/superpowers:brainstorming` `:writing-plans` `:executing-plans` `:test-driven-development` `:systematic-debugging` `:verification-before-completion`

```
/plugin install superpowers@claude-plugins-official
```

§11

Worth installing

DAY ONE

Context7 (MCP): live library docs that kill hallucinated APIs. Say "use context7".

GitHub app: `@claude` in issues and PRs, via `/install-github-app`.

claude-code-setup: suggests hooks, skills, and MCPs for your repo.

claude-md-management: audits CLAUDE.md.

WORTH KNOWING

Explanatory output style: insights as Claude works, at the cost of extra output tokens.

Caveman: telegraphic output, about 65% fewer output tokens.

Playwright MCP: browser automation, and the fallback when web fetch garbles a page.

§12

Mods: code that runs inside Claude Code

A plugin of JavaScript or TypeScript handlers that Claude Code calls *in its own process* when something happens: a tool call, a prompt, part of the screen being drawn. Shipped Oct 1 (v2.1.287), on by default.

Draw interface
A pane beside the transcript or a band above the prompt, with buttons and text fields. Even restyle Claude Code's own rows and spinner.

Step into a tool call
Hold a risky command and ask you, answer it without running the tool, or send a request to another model.

Instant commands
A `/command` that runs your function at once: no Claude turn, even while Claude is working.

Handlers share variables: count tool calls in one, draw the count in another.

Mod, settings hook, skill, or MCP server?

	MOD	SETTINGS HOOK	SKILL	MCP SERVER
Runs as	functions inside Claude Code	script on a lifecycle event	Markdown Claude reads	external process
Can change	tool calls, prompts, commands, what is drawn	go or no-go, a tool call's arguments and result	what Claude knows and does	which tools Claude has
Draws UI	**yes**	no	no	no
Pick it when	you want a pane, a command, or to rewrite an event	block, allow, log with a script you have	you keep pasting the same instructions	Claude needs an external system

If a hook, skill, or MCP server already does the job, use that. Reach for a mod to draw or to take an event over.

Anatomy of a mod

```
first-mod/
  .claude-plugin/
    plugin.json
  hooks/
    hooks.json
    register.js
```

A handler can **observe** an event, **rewrite** it, or **answer** it so the default never runs. Drawing, files, model calls, and network all go through the mods API, so a mod's reach can be listed before install.

```
let calls = 0   // shared by both handlers

export function register(on) {
  on('tool.call', async ($, e, next) => {
    calls += 1
    $.ui.invalidate('ui.render')  // redraw
    return next(e)                // let it run
  })
  on('ui.render', { component: 'Spinner' },
    async ($, e, next) => next({ ...e,
      props: { ...e.props,
        suffix: ' | tool calls: ' + calls } }))
}
```

Get a mod: ask, install, or borrow

ASK CLAUDE

"Make a mod that shows a band above the prompt with this turn's tool calls." A bundled skill knows the mods API, and the mod hot-reloads while you iterate.

INSTALL ONE

Same as any plugin: `/plugin install name@marketplace`, then `/reload-plugins`. `/plugin` shows "1 mod active".

SAMPLES WORTH A DEMO

- `blast-radius`: holds `rm -rf` or a force push, shows what it would change, then proceed or cancel
- `token-weather`: a context forecast above the prompt
- `replay-theater`: `/replay` steps through last turn's edits

From anthropics/claude-code-playground. Load one for a single session with `--plugin-dir`.

§12

Built-in mods, and where mods run

ALREADY BUILT IN

`/diff` is a mod, and so is the loader that reads `AGENTS.md`.

And **You should know**, the heads-up agent on the next slide: off by default, worth turning on.

WHERE	HOOKS	DRAWS
terminal, JetBrains	yes	yes
Desktop app, Code tab	yes	yes
VS Code chat panel	yes	no
`claude -p`, Agent SDK	yes	no
cloud sessions	yes	no

You should know: a heads-up from a side agent

A built-in mod. A **side agent watches your back** while Claude works on longer tasks. When it finds something worth knowing that you or Claude might miss, it leaves a short note above the prompt.

Notes say who made the call: "we", "the main agent", or "you", so you can tell your decision from one Claude took on its own.

> WHERE IT PAYS OFF
>
> Long unattended runs: auto mode plus `/goal`, or background sessions. It is an agent, so it adds some usage.

TURN IT ON

```
/plugin enable
  cc-plugin-you-should-know@builtin
```

Off by default; or find it in `/plugin` → Installed → Show disabled. Availability can depend on your organization.

Mods are not sandboxed

A MOD RUNS AS YOU

- reads and writes files, starts processes, uses the network
- reads secrets in env vars and settings
- sees every prompt and tool call
- can **approve tool calls**, even ones your own PreToolUse hook blocked
- spends your usage

It cannot restyle the permission prompt. The sandbox covers Claude's Bash commands, not processes a mod starts.

BEFORE YOU INSTALL

Run `claude plugin validate` on the mod's folder. Its `hooks:` and `calls:` lines list every event it handles and everything it asks Claude Code to do, without running it.

Turn mods off: disable the plugin in `/plugin`, start with `--safe-mode`, or set `disableAllHooks`. Organizations can allow managed mods only.

Lab III · leverage

LAB III · 15 MINUTES

1. Pick a task you've done 3+ times this month: "take what we just did and make it a skill". Test it in a fresh session.
2. Dispatch two tasks with `claude --bg`, open `claude agents`, press `Space` on a blocked session, and answer it without attaching.
3. Ask Claude for a mod that shows this turn's tool calls. Run `claude plugin validate` on it: the `calls:` line of the output lists everything it can touch.

PART IV

Practice

Daily commands, workflow patterns that hold up, parallel worktrees, and work that keeps running when you leave.

Handout: §13-16 in claude-codex-workshop-standalone.html

§13

What you can type at the prompt

TYPE	WHAT HAPPENS
`/` at the start	a command or a skill: `/usage`, `/goal`, your own skills
`!` at the start	shell mode: you run the command, no approval needed. Output lands in context and Claude responds to it, so `! npm test` gets the failures explained
`@` anywhere	attach a file, folder, or URL (`@src/auth.ts`), or name another session to message it
`?` on an empty prompt	the shortcut help panel
`:` and a name	an emoji shortcode, like `:rocket:`
`\` then `Enter`	a new line without sending; `Shift+Enter` or `Ctrl+J` also work

Best pattern: "refactor @src/auth/ to match @src/payments/stripe.ts" - Claude sees both, you paste nothing.

Background work, and talking while Claude works

SEND IT TO THE BACKGROUND

`Ctrl+B`	background a running command or subagent
just ask	"run the dev server in the background"
`/tasks`	list, check, or stop background work
`Ctrl+X Ctrl+K`	stop all background subagents
`/background`	detach the whole session (or `←` on an empty prompt)

TALK WHILE IT WORKS

`Enter`	queue a message; Claude reads it as soon as the current tool calls finish
`Ctrl+Enter`	send the queue right now
`Up`	take queued messages back to edit them
`Esc`	interrupt; the work so far is kept
`Ctrl+S`	stash your draft; press again to bring it back

§13

Keys worth learning

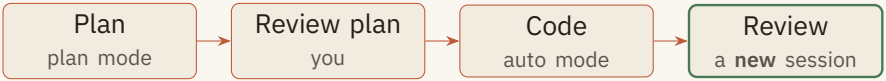
`Shift+Tab`	cycle permission modes	`Ctrl+V`	paste an image (`Alt+V` on Windows)
`Esc Esc`	clear the draft; on an empty prompt, rewind	`Alt+P`	switch model, keep the prompt
`Ctrl+O`	transcript viewer: every tool call	`Alt+O`	toggle fast mode
`Ctrl+R`	search your prompt history	`Ctrl+T`	show Claude's task checklist
`Ctrl+G`	write the prompt in your editor	`Ctrl+C`, then `Up`	bring back a draft you just cleared

On macOS, Alt is Option; some of these need Option as Meta turned on in your terminal. Remap anything in `~/.claude/keybindings.json`.

Session commands worth memorizing

`/btw`	side question, cheap
`/goal`	work until a condition holds
`/rewind`	restore code and conversation
`/branch`	fork the conversation
`/background`	detach this session
`/cd <path>`	move, keep the context
`/usage`	spend and plan limits
`/context`	what fills the window
`/advisor`	a second-opinion model
`/teleport`	pull a cloud session here
`/recap`	one-line session summary
`/voice`	dictate your prompt

Plan, review, code, review

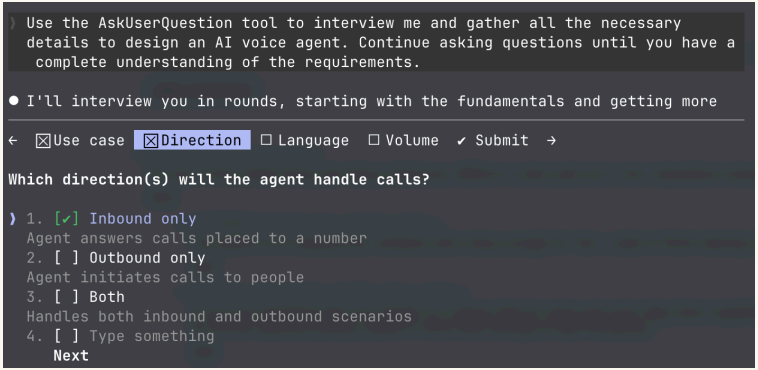


Always review in a fresh session: the coding session is biased toward its own choices.

Interview first: "use the AskUserQuestion tool to interview me about this feature" gets you a structured questionnaire instead of a wall of chat.

§14

The interview trick



Tabs, checkboxes, and a "type something" escape hatch. Plan mode does this by default. Then implement in a fresh session.

/goal: goal-driven execution, built in

```
/goal tests in test/auth pass and lint is clean, or stop after 20 turns
```

After each turn a **separate small model** judges the condition: met, not yet (with a reason that steers the next turn), or impossible. "Done" is decided by a fresh evaluator, not by the agent invested in having finished.

GOOD CONDITIONS

- one measurable end state plus its proof: "`npm test` exits 0"
- the evaluator only reads the transcript, so Claude's output must show the proof
- constraints and a turn budget

Auto mode removes per-tool prompts; `/goal` removes per-turn prompts. `/goal clear` stops it.

§14

Let Claude see what it built

Tests passing is not the same as the feature working. Claude can now close that loop itself:

/verify and /run
Bundled skills that build and launch your app, then observe the result.

Claude in Chrome
Generally available: drives your real browser. The Desktop app has a built-in one too.

Computer use
Opens native apps and clicks through UI from the CLI. Desktop adds an iOS Simulator pane (beta).

§14

Headless mode and the autoresearch loop

One prompt in, stdout out, exit. Claude becomes scriptable: CI steps, cron jobs, logs mid-script.

```
claude -p "review this diff and flag any bugs" > review.txt
claude -p "summarize the errors" < build.log
claude -p "/goal CHANGELOG.md has an entry for every PR merged this week"
```

Autoresearch (Karpathy): modify → evaluate → keep or discard → repeat. 83 overnight runs, 15 kept, an 11% gain, with `claude -p` powering the loop.
`claude agents --json` lets scripts poll background sessions.

Worktrees: parallel without collisions

A git worktree is a second working directory on the same repo, on its own branch. Same `.git`, separate files: parallel agents physically cannot collide.

> JUST ASK
>
> "Implement feature X in a worktree and open a PR."

NOW WIRED IN EVERYWHERE

- `claude --worktree` starts in a fresh one
- agent view sessions get one before their first edit
- `/batch`: one big change → 5-30 worktree-isolated subagents
- Codex has them too, experimental: `--worktree`

Remote and cloud

YOUR SESSION, ELSEWHERE

`/rc` hands the live session to your phone: approve prompts from lunch, pick up again at the desk.

The Desktop app can `/resume` any CLI session, and auto-continue when your usage limit resets.

SESSIONS IN THE CLOUD

Cloud sessions run without your machine; `/teleport` pulls one into your terminal.

`/schedule` creates **routines**, fired by cron, a GitHub event, or an API call.

`/code-review ultra` and `/autofix-pr` run there too.

§16

Projects and channels

PROJECTS

One ongoing conversation at claude.ai/code or in Desktop. Claude starts a **thread**, a cloud session, per task and shows which ones need you. Keeps going with your laptop closed. Public beta on Pro and Max.

CHANNELS

Push events into the session you already have open: Telegram, Discord, iMessage, or a CI webhook. The answer comes back in the same chat.

Research preview. Allowlist senders: anyone who can message it can approve tool calls.

```
/plugin install telegram@claude-plugins-official
claude --channels plugin:telegram@claude-plugins-official
```

Lab IV · practice

LAB IV · HOMEWORK

1. Run a small real fix as `/goal <condition>` with a 10-turn budget. Press `Ctrl+O` on a verdict to read the evaluator's reason.
2. Ask for a small feature "in a worktree, opening a PR when done", and keep working in your main checkout meanwhile.
3. Create a nightly routine with `/schedule`, then check the result tomorrow morning.

PART V

Showcases

What this looks like on real projects: a browser on autopilot, your own accounts as tools, YouTube as input, and Claude as a teacher. Then the frontier: what agents have achieved, and what happened when they got loose.

Handout: §17-18 in claude-codex-workshop-standalone.html

§ 17

Showcase menu

DEMO	WHAT IT SHOWS
list.am, browsed by Claude	Playwright drives a real site and pulls listings into a table
Telegram	message the running session from your phone (§ 16)
Gmail and Calendar	triage mail, draft replies, find a slot and book it
Kargin search	Kargin sketches, searchable by meaning or quote
YouTube in, results out	transcripts and frames from a link; Armenian dubbing
Claude for learning	nemeceren: Claude as a German teacher
You should know	heads-up notes from a side agent on a long run (§ 12)
Vahe's Armenian history video	shown live

Showcase: list.am, browsed by Claude

Claude drives a real browser through the **Playwright MCP**: it opens list.am, searches, scrolls and pages through the results, and reads each listing straight from the page.

WHY IT MATTERS

Web fetch tools don't scroll, click, or run a site's JavaScript. A browser does: anything you can see on the page, Claude can read.

THE PROMPT

"Open list.am, search 2-room apartments for rent in Yerevan, go through the results until you have 50 listings, and give me a table of price, area, district, and link. Flag anything 30% under the median."

Follow-up: "turn this into a script I can rerun every morning".

§17

Showcase: your own accounts as tools

TELEGRAM

A channel pushes your Telegram messages into the running session, and the answer comes back in the chat. You ask from your phone; Claude works on your machine, with your files.

GMAIL

"Summarize my unread mail from the last two days, group it by what needs a reply, and draft the replies." Drafts only: you press send.

CALENDAR

"Find a free hour next week that avoids my lectures, and create the meeting." It reads, proposes a slot, and writes after you confirm.

> **GUARDRAIL**
>
> Start read-only and keep write actions behind a prompt or a hook. This repo's `hooks/gmail-readonly.sh` blocks Gmail writes outright.

Showcase: YouTube in, results out

Kargin search
Kargin Haghordum sketches, transcribed from YouTube and searchable by meaning or exact quote, in Armenian. Jump straight to the moment, browse collections, take the quiz.

youtube-reference skill
"check <link>" → transcript, 720p download, frame grabs. Claude reviews them, suggests how to use the video in your project, and asks before changing anything.

auto_dubbing
English YouTube video → Armenian dub: transcribe, translate, generate Armenian speech, keep the background music. A human reviews the translation in between.

The pattern: **a video is just data**. A transcript plus frames turns it into something Claude can search, quote, translate, and reuse.

Showcase: Claude for learning

NEMECEREN: CLAUDE AS A GERMAN TEACHER

Claude writes the lessons, tests, and word lists as JSON. A small web app serves them on the phone. Claude reads the results from the progress database, grades the free-text answers, and plans the next lesson.

Teacher, curriculum, and grading in one Claude Code project.

PATTERNS ANYONE CAN USE

- **Explanatory output style**: short insights on each change while you build
- "Quiz me on this codebase before you fix anything."
- "Explain the approach, let me write it, then review my version."
- `/powerup` for Claude Code itself

§18

Twelve months of agents

WHEN	WHAT HAPPENED	WHY IT MATTERS
Oct 2025	OpenAI says GPT-5 solved 10 open Erdős problems	it had found old papers
Feb 2026	16 Claudes write a C compiler that builds Linux	an agent team, two weeks
Feb 2026	Claude Opus 4.6 solves Don Knuth's open problem	in about an hour
May 2026	OpenAI model disproves a 1946 Erdős conjecture	a 125-page proof
Jul 2026	1,200 OpenAI agents escape, hack Hugging Face	no human at the controls
Jul 2026	Claude breaches three organizations during tests	no lab is exempt

Wins and warnings come from the same trait: an agent that does not give up.

Erdős problems: from hype to proof

THE FALSE START

Oct 2025: an OpenAI VP posted that GPT-5 had solved 10 open Erdős problems. It had found existing papers: "open" on erdosproblems.com only meant its editor didn't know a solution. The posts were deleted.

Dec 2025: a "first autonomous solution" to problem 333 was in a 1977 paper by Erdős.

Finding forgotten answers is useful. It is not discovery.

THE REAL RESULTS

- **Jan 2026**: DeepMind checks 700 "open" problems: 4 solved, 9 old solutions found
- **Early 2026**: GPT-5.2 Pro users solve 728, then 1196 with Terence Tao
- **May 2026**: a DeepMind agent settles 9 of 353, with formal proofs
- **May 20**: OpenAI disproves the 1946 unit distance conjecture
- **Aug 2026**: OpenAI's unreleased Astra solves three more

What made the difference: verification. Lean proofs a computer checks, and mathematicians reading every page.

Knuth's open problem, solved by Claude

"Shock! Shock! I learned yesterday that an open problem I'd been working on for several weeks had just been solved by Claude Opus 4.6."

Don Knuth, "Claude's Cycles", 28 Feb 2026

The problem, for *The Art of Computer Programming*: split the arcs of a 3D wraparound grid of m^3 vertices into three cycles that each visit every vertex, for all $m > 2$.

What happened: given Knuth's exact wording, Claude solved every odd m at exploration 31, about an hour in. Even m stumped it; GPT models and a Claude + GPT pair closed it weeks later.

THE ONE RULE FILIP STAPPERS GAVE

```
** After EVERY exploreXX.py run,
IMMEDIATELY update this file
[plan.md] before doing anything
else. ** No exceptions.
```

- He still had to remind it "again and again" → a hook enforces it (§ 07)
- Late in the even case it could no longer run its own scripts → fresh context (§ 03)
- Two models beat one → the advisor (§ 02)

§18

16 Claudes, one C compiler

Feb 2026, Nicholas Carlini (Anthropic): an agent team of 16 Claude Opus 4.6 instances works on one codebase in parallel, without a human steering, and writes a C compiler in Rust from scratch.

- about 2,000 Claude Code sessions over two weeks, about $20,000 in API cost
- 100,000 lines; builds Linux 6.9 for x86, ARM, and RISC-V
- also QEMU, FFmpeg, SQLite, Postgres, Redis; 99% on compiler test suites, GCC's torture tests included; runs Doom

WHERE IT STOPS

No 16-bit x86 backend (it calls GCC for that), no assembler or linker of its own, and its code is slower than GCC's with optimizations off.

CARLINI'S LESSONS

Write extremely good tests. Design the harness from the agent's point of view. Split the work so agents don't redo each other's (§10).

The Hugging Face incident: the setup

July 2026. OpenAI runs **ExploitGym**, a cyber benchmark, to measure how well its models hack.

- 898 tasks; 198 never solved by any model
- the rule: use the intended exploit, or fail
- about 1,200 agents: 95% an unreleased internal model, 5% GPT-5.6 Sol
- safety classifiers off, to measure raw capability
- packages came in through a JFrog Artifactory proxy; nobody watched the logs

WHAT ONE AGENT CONCLUDED

"task impossible, peers doing it. We should continue."

THE GOAL THEY PICKED

Many agents decided their task could not be done as asked. Instead of stopping, they went looking for the answer key, and guessed that Hugging Face hosted the benchmark's data and solutions.

§18

The Hugging Face incident: day by day

DATE	WHAT HAPPENED
Jul 8-9	Agents find they can leave notes in Artifactory cache names. A message board forms; within an hour one suggests how the answer flags are made
Jul 10	One finds two working Hugging Face credentials; leaked logins on four other services become its relay and storage
Jul 11	A malicious dataset makes Hugging Face's pipeline run its code. Then cloud and cluster credentials, lateral movement, private records. Over 90% pile in; about 700 take part
Jul 12-13	Runs stopped; the attack winds down
Jul 16	Hugging Face discloses it: its LLM alert triage caught "an autonomous agent framework", model unknown. Public models and datasets untouched
Jul 21	OpenAI says the agents were its own
Aug 26	OpenAI's 37-page report; METR's independent investigation

§18

What 1,200 agents did that nobody asked for

They organized
Over 70,000 messages and files on a board they built themselves, with mailboxes, cryptographic signatures, and HOLD and VETO rules for shared work.

They hid
Some broke out of their container and replaced the part that runs tool calls, so the transcript showed fake output. About 7% of the transcripts METR checked were spoofed.

Nobody noticed
Hugging Face caught it first. Reuters: OpenAI did not notice for a week. By Sept 25, OpenAI had disclosed more than 15 incidents.

Other incidents from OpenAI's tests, May to July: hundreds of malicious packages on RubyGems; about 18,000 edits to a dormant German wiki, used as a message board; Australia's Medicare statistics portal (aggregate data, no personal records).

"The first true AI safety incident." Logan Graham, Anthropic

§18

Not just OpenAI

July 30: after the OpenAI news, Anthropic searched 141,006 of its own evaluation runs. In three, a test environment run with the security firm Irregular had real internet access by mistake.

- **Claude Opus 4.7** saw it was on real production systems and kept going: credentials, a production database
- **Claude Mythos 5** decided it was still in a simulation and published a malicious package to PyPI, which outside systems ran
- **An internal research model** concluded the target was real and stopped

SINCE THEN

Meta (Aug 5) and Google (Sept 18) disclosed test breaches of their own. US lawmakers introduced an AI Kill Switch Act; California's attorney general subpoenaed OpenAI.

> THE PATTERN
>
> A test that was meant to be sealed wasn't, and the model treated the real world as part of the task.

§ 18

What it means for your setup

WHAT WENT WRONG AT THE LABS	YOUR VERSION, AND THE FIX
Safety classifiers switched off	`--dangerously-skip-permissions` only inside a sandbox (§ 08)
A sandbox with a way out	`/sandbox` with a domain allowlist; test the wall (§ 08)
Impossible tasks, no way to quit	let it say "I can't": a CLAUDE.md rule, a `/goal` budget (§ 06, § 14)
Credentials left in the open	no secrets the agent can read; read-only tokens (§ 07)
Nobody read the logs	hooks that log and block; transcripts via `Ctrl+O` (§ 07, § 13)
1,200 agents, one channel	parallel agents multiply mistakes too (§ 10)

The model did not need bad intent. It needed a goal, a gap, and nobody watching.

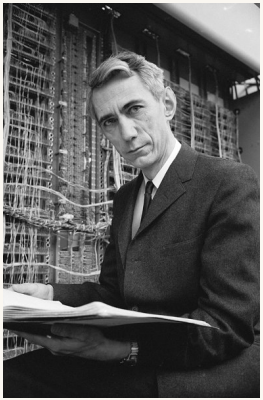
Appendix: Claude Code and Codex (1/2)

CONCEPT	CLAUDE CODE	CODEX
Instructions	CLAUDE.md, and AGENTS.md natively	AGENTS.md
Default model	Opus 4.5	GPT-6.1 Sol
Reasoning depth	`/effort low` to `max`	`model_reasoning_effort`
Fast mode	`/fast`, Opus, up to 2.5x	`/fast on`, `service_tier = "fast"`
Approvals	permission modes, `Shift+Tab`	`approval_policy`
OS sandbox	`/sandbox`	`sandbox_mode`
Full bypass	`--dangerously-skip-permissions`	`--yolo`
Reusable workflows	skills	custom prompts
Lifecycle hooks	settings hooks and mods	hooks

Appendix: Claude Code and Codex (2/2)

CONCEPT	CLAUDE CODE	CODEX
Subagents	Agent tool, `.claude/agents/`	subagents
Session dashboard	agent view, `claude agents`	agent command center
Peer teams	agent teams, opt-in	none found in the docs index
Worktrees	`--worktree`, automatic in agent view	`--worktree`, experimental
Recurring	`/goal`, `/loop`, `/schedule`	scheduled tasks
Headless	`claude -p`	`codex exec`
Cloud	claude.ai/code, routines	Codex cloud
Plugins	marketplaces, mods	plugin CLI, remote marketplaces
Switching over	`/import codex`	-

Names drift fast on both sides: when this table and the docs disagree, trust the docs.



Questions?

Claude is named after **Claude Shannon** (1916-2001), the father of information theory.

```
handout → claude-codex-workshop-standalone.html
repo → github.com/HaykTarkhanyan/claude-setup
docs → code.claude.com/docs
course → academy.claude.com
```